# HiddenGem

## Service Layer Design

REST API endpoints | Request & response samples | Page-to-endpoint diagram

Prepared by: Cheryl Wang Smith
Course: SWDV 691: SOFTWARE DEVELOPMENT CAPSTONE 1W 26/SP2
Date: March 22, 2026

# What Changed From the Original Design

The core discovery-and-sharing experience is unchanged. The social graph (follow/unfollow) has been moved to a stretch feature to keep the MVP tightly scoped.

# Section 1: Design Overview

The HiddenGem service layer is a RESTful JSON API built with Node.js and Express, deployed on Railway. It sits between the React front-end and the PostgreSQL database and is the only process that reads from or writes to the database. The front-end communicates exclusively through these endpoints over HTTPS.

All endpoints follow REST conventions: resources are identified by nouns in the URL path, HTTP verbs express the intended operation, and HTTP status codes communicate the outcome. All request and response bodies are JSON. Authentication is JWT-based: the client sends the token in the Authorization header as Bearer <token>.

## Base URL

```
https://api.hiddengem.app/v1
```

## HTTP verb conventions

- GET -- retrieve a resource or collection. Never modifies data.
- POST -- create a new resource or perform a non-idempotent action (e.g. login).
- PATCH -- partial update of an existing resource (only fields provided are changed).
- DELETE -- remove a resource.

## Standard error envelope

All error responses use this shape regardless of the endpoint:

```
{
  "error": {
    "code":    "VALIDATION_ERROR",
    "message": "Human-readable description of what went wrong.",
    "fields":  { "email": "Must be a valid email address." }
  }
}
```

The fields object is only present for validation errors and maps field names to their specific error messages.

## Common error codes

| Status | Meaning | When returned |
|---|---|---|
| 400 | Bad Request | Request body or query params failed validation. |

| | | |
|---|---|---|
| 401 | Unauthorized | JWT missing, expired, or invalid. |
| 403 | Forbidden | Authenticated user does not own the resource. |
| 404 | Not Found | Resource with the given ID does not exist. |
| 409 | Conflict | Unique constraint violation (e.g. email already registered). |
| 422 | Unprocessable Entity | Request was well-formed but semantically invalid (e.g. privacy value not in allowed enum). |
| 500 | Internal Server Error | Unexpected server-side failure. |

# Section 2: Authentication Endpoints

These endpoints handle account creation, login, and email verification. No JWT is required to call them. On success, POST /auth/register and POST /auth/login both return a JWT that the client stores in memory and sends with every subsequent authenticated request.

| POST | /auth/register | Create a new account | Public |
|---|---|---|---|

Purpose: Maya opens the Sign Up page and submits her name, email, and password. This creates a new row in the users table with is_verified = FALSE and sends a verification email via SendGrid. The response returns a JWT so Maya can immediately browse but cannot post until she verifies.

**Request body**

```
{
  "display_name": "Maya Chen",
  "username":     "mayachen",
  "email":        "maya@example.com",
  "password":     "S3cure!Pass"
}
```

**Success response -- 201 Created**

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id":           "a1b2c3d4-...",
    "username":     "mayachen",
    "display_name": "Maya Chen",
    "is_verified":  false
  }
}
```

**Error -- 409 Conflict (email already exists)**

```
{
  "error": {
    "code":    "EMAIL_TAKEN",
    "message": "An account with this email already exists."
  }
}
```

**Error -- 400 Validation**

```
{
  "error": {
    "code":    "VALIDATION_ERROR",
```

```
    "message": "Request body failed validation.",
    "fields": {
      "password": "Must be at least 8 characters.",
      "username":  "May only contain letters, numbers, and underscores."
    }
  }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 201 | Created | Account created. JWT returned. |
| 400 | Bad Request | Validation failed (missing field, bad format, weak password). |
| 409 | Conflict | Email or username already registered. |

| POST | /auth/login | Sign in to existing account | Public |
|---|---|---|---|

Purpose: James returns to the app and submits his email and password on the Sign In page. On success the API verifies the bcrypt hash and returns a fresh JWT. The client stores this token and includes it in all subsequent requests.

### Request body

```
{
  "email":    "james@example.com",
  "password": "MyPass123!"
}
```

### Success response -- 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id":           "b2c3d4e5-...",
    "username":     "jamesokafor",
    "display_name": "James Okafor",
    "avatar_url":
"https://res.cloudinary.com/hiddengem/image/upload/v1/avatars/james.jpg",
    "is_verified":  true
  }
}
```

### Error -- 401 Unauthorized (wrong credentials)

```
{
  "error": {
    "code":    "INVALID_CREDENTIALS",
    "message": "Email or password is incorrect."
  }
```

```
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Login successful. JWT returned. |
| 400 | Bad Request | Email or password field missing. |
| 401 | Unauthorized | Credentials do not match any account. |

| POST | /auth/verify-email | Confirm email address | Public |
|---|---|---|---|

Purpose: Maya clicks the verification link in her email. The link contains a short-lived token. This endpoint validates it and sets is_verified = TRUE on her user row.

#### Request body

```
{
  "token": "abc123verifytoken"
}
```

#### Success response -- 200 OK

```
{
  "message": "Email verified successfully."
}
```

#### Error -- 400 (token expired or invalid)

```
{
  "error": {
    "code":    "INVALID_VERIFY_TOKEN",
    "message": "Verification link is invalid or has expired. Please request a new
one."
  }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Email verified. |
| 400 | Bad Request | Token missing, malformed, or expired. |

| POST | /auth/resend-verification | Resend verification email | Public |
|---|---|---|---|

Purpose: Maya did not receive her email and clicks "Resend Email" on the Email Verification screen.

### Request body

```
{ "email": "maya@example.com" }
```

### Success response -- 200 OK

```
{ "message": "Verification email resent." }
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Email sent (or silently ignored if already verified). |
| 400 | Bad Request | Email field missing or malformed. |

# Section 3: User Endpoints

These endpoints expose user profile data and allow authenticated users to update their own profile.

| GET | /users/:username | Get a public user profile | Auth required |
|---|---|---|---|

Purpose: Sofia visits /profile/jamesokafor. The front-end fetches James's profile including his gem count, follower/following counts, and whether Sofia is already following him (used to render the Follow vs Following button state).

**Success response -- 200 OK**

```
{
  "id":            "b2c3d4e5-...",
  "username":      "jamesokafor",
  "display_name":  "James Okafor",
  "bio":           "15-year neighborhood veteran. Sharing hidden gems in
Brooklyn.",
  "avatar_url":    "https://res.cloudinary.com/hiddengem/...",
  "gem_count":     127,
  "follower_count":1240,
  "following_count":89,
  "is_following":  false,
  "created_at":    "2025-03-01T12:00:00Z"
}
```

**Error -- 404 Not Found**

```
{ "error": { "code": "USER_NOT_FOUND", "message": "No user with that username
exists." } }
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Profile returned. |
| 404 | Not Found | Username does not exist. |

| PATCH | /users/me | Update own profile | Auth required |
|---|---|---|---|

Purpose: James opens his Edit Profile modal and changes his bio and avatar photo. Only fields present in the request body are updated (partial update). Avatar photo upload is handled separately via POST /uploads/avatar.

**Request body (all fields optional)**

```
{
  "display_name": "James Okafor",
  "bio":          "Local gem hunter. Brooklyn born and raised.",
  "avatar_url":
"https://res.cloudinary.com/hiddengem/image/upload/v1/avatars/james2.jpg"
}
```

### Success response -- 200 OK

```
{
  "id":           "b2c3d4e5-...",
  "username":     "jamesokafor",
  "display_name": "James Okafor",
  "bio":          "Local gem hunter. Brooklyn born and raised.",
  "avatar_url":   "https://res.cloudinary.com/hiddengem/..."
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Profile updated. |
| 400 | Bad Request | Validation error on a field. |
| 401 | Unauthorized | No valid JWT provided. |

# Section 4: Gem Endpoints

Gems are the core content resource of HiddenGem. These endpoints power the Discover feed, the map view, and the gem detail page.

| GET | /gems | List / discover gems | Auth required |
|---|---|---|---|

Purpose: Sofia opens the Discover page. The front-end calls this endpoint to populate both the feed card grid and the map pins. Accepts query parameters to support location filtering, category filtering, tag filtering, and feed vs. following mode.

### Query parameters

```
lat=40.7128          (optional) center latitude for distance sort / radius filter
lng=-74.0060         (optional) center longitude
radius=10            (optional) km radius, default 50
category=Nature      (optional) filter by category enum
tag=waterfall        (optional) filter by tag name
feed=following       (optional) "following" returns only gems from followed users
sort=newest          (optional) newest|nearest|most_saved, default newest
page=1               (optional) pagination page, default 1
limit=20             (optional) results per page, default 20
```

### Success response -- 200 OK

```
{
  "gems": [
    {
      "id":           "c3d4e5f6-...",
      "name":         "Hidden Cascade Behind the Old Mill",
      "category":     "Nature",
      "latitude":     40.7282,
      "longitude":    -73.7949,
      "location_label":"Mill Creek Park, Riverside County",
      "cover_photo":
"https://res.cloudinary.com/hiddengem/image/upload/v1/gems/xyz.jpg",
      "save_count":   24,
      "distance_km":  2.3,
      "tags":         ["waterfall", "nature", "hiking"],
      "author": {
        "username":    "johndoe_explorer",
        "display_name":"John Doe",
        "avatar_url":  "https://res.cloudinary.com/hiddengem/..."
      },
      "created_at":  "2026-03-10T14:30:00Z"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 143 }
```

```
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Gem list returned (may be empty array). |
| 400 | Bad Request | Invalid query param (e.g. category not in enum, radius not numeric). |
| 401 | Unauthorized | JWT required for feed=following. |

**GET** `/gems/:id` Get a single gem detail Auth required

Purpose: Sofia clicks a map pin or feed card and navigates to /gems/:id. The full detail page needs the complete gem data including all photos, all tags, whether the current user has saved it, and the first page of comments.

### Success response -- 200 OK

```
{
  "id":             "c3d4e5f6-...",
  "name":           "Hidden Cascade Behind the Old Mill",
  "description":    "Stumbled upon this incredible hidden waterfall...",
  "category":       "Nature",
  "latitude":       40.7282,
  "longitude":      -73.7949,
  "location_label": "Mill Creek Park, Riverside County",
  "privacy":        "public",
  "view_count":     247,
  "save_count":     24,
  "is_saved":       false,
  "tags":           ["waterfall", "nature", "hiking"],
  "photos": [
    { "url": "https://res.cloudinary.com/.../gem_cover.jpg", "display_order": 0 },
    { "url": "https://res.cloudinary.com/.../gem_2.jpg",     "display_order": 1 }
  ],
  "author": {
    "username":    "johndoe_explorer",
    "display_name":"John Doe",
    "avatar_url":  "https://res.cloudinary.com/..."
  },
  "created_at": "2026-03-10T14:30:00Z"
}
```

### Error -- 404 Not Found

```
{ "error": { "code": "GEM_NOT_FOUND", "message": "Gem not found." } }
```

| Status | Meaning | When returned |
|---|---|---|

| | | |
|---|---|---|
| 200 | OK | Full gem returned. view_count incremented server-side. |
| 401 | Unauthorized | Required if gem privacy is followers_only. |
| 403 | Forbidden | Gem is private and current user is not the owner. |
| 404 | Not Found | Gem does not exist or is flagged. |

**POST** `/gems` Submit a new gem Auth required

Purpose: James fills out the Create page and clicks "Publish Hidden Gem". Photos must be uploaded first via POST /uploads/photo which returns a Cloudinary URL. That URL is included in this request body. A verified account is required to submit.

### Request body

```
{
  "name":           "Secret Waterfall Trail",
  "description":    "Hidden behind the old mill...",
  "category":       "Nature",
  "latitude":       40.7282,
  "longitude":      -73.7949,
  "location_label": "Mill Creek Park",
  "privacy":        "public",
  "tags":           ["waterfall", "hiking", "nature"],
  "photos": [
    { "url": "https://res.cloudinary.com/.../gem_1.jpg", "display_order": 0 },
    { "url": "https://res.cloudinary.com/.../gem_2.jpg", "display_order": 1 }
  ]
}
```

### Success response -- 201 Created

```
{
  "id":        "d4e5f6g7-...",
  "name":      "Secret Waterfall Trail",
  "created_at":"2026-03-22T10:15:00Z"
}
```

### Error -- 403 Forbidden (unverified account)

```
{
  "error": {
    "code":    "EMAIL_NOT_VERIFIED",
    "message": "You must verify your email address before submitting a gem."
  }
}
```

### Error -- 422 Unprocessable Entity (invalid privacy value)

```
{
```

```
  "error": {
    "code":    "VALIDATION_ERROR",
    "message": "Request body failed validation.",
    "fields":  { "privacy": "Must be one of: public, followers_only, private." }
  }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 201 | Created | Gem created. Pins appear on map immediately. |
| 400 | Bad Request | Missing required field or bad coordinates. |
| 401 | Unauthorized | No valid JWT. |
| 403 | Forbidden | Account email not yet verified. |
| 422 | Unprocessable Entity | privacy or category value not in allowed enum. |

**PATCH** `/gems/:id` Update an existing gem Auth required

Purpose: James opens the Edit Post page for one of his submissions and changes the description or privacy level. Only the fields included in the request body are updated. Only the gem owner may call this endpoint.

### Request body (all fields optional)

```
{
  "description": "Updated: best visited in early morning when mist rises.",
  "privacy":     "followers_only"
}
```

### Success response -- 200 OK

```
{
  "id":          "d4e5f6g7-...",
  "description": "Updated: best visited in early morning when mist rises.",
  "privacy":     "followers_only",
  "updated_at":  "2026-03-22T11:00:00Z"
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Gem updated. |
| 400 | Bad Request | Validation error on a field. |
| 401 | Unauthorized | No valid JWT. |
| 403 | Forbidden | Current user does not own this gem. |
| 404 | Not Found | Gem does not exist. |

| DELETE | /gems/:id | Delete a gem | Auth required |
|---|---|---|---|

Purpose: James decides to remove a submission from the Edit Post page. Deleting a gem cascades to delete all associated photos, tags, comments, and saves rows in the database.

**Success response -- 204 No Content**

```
(empty body)
```

| Status | Meaning | When returned |
|---|---|---|
| 204 | No Content | Gem and all related records deleted. |
| 401 | Unauthorized | No valid JWT. |
| 403 | Forbidden | Current user does not own this gem. |
| 404 | Not Found | Gem does not exist. |

| POST | /gems/:id/flag | Flag a gem as inappropriate | Auth required |
|---|---|---|---|

Purpose: Any authenticated user can flag a gem from the gem detail page action bar. This sets is_flagged = TRUE on the gem, hiding it from all public queries until an admin reviews it.

**Success response -- 200 OK**

```
{ "message": "Gem reported. It will be reviewed by our team." }
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Gem flagged. |
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | Gem does not exist. |

# Section 5: Photo Upload Endpoint

Photo upload is handled as a two-step process. The client first uploads the binary file to this endpoint, which forwards it to Cloudinary and returns the CDN URL. The client then includes that URL in the gem create or update request body. This keeps binary data out of the gem endpoints and decouples media handling from content logic.

| POST | /uploads/photo | Upload a photo to Cloudinary | Auth required |
|---|---|---|---|

Purpose: James selects a photo on the Create page. The front-end sends it as multipart/form-data. The API uploads it to Cloudinary and returns the resulting CDN URL. James's client stores this URL and includes it when submitting the gem.

**Request (multipart/form-data)**

```
Content-Type: multipart/form-data

file: <binary image data>   PNG, JPG, or GIF, max 10MB
```

**Success response -- 201 Created**

```
{
  "url": "https://res.cloudinary.com/hiddengem/image/upload/v1/gems/abc123.jpg"
}
```

**Error -- 400 (file too large)**

```
{
  "error": {
    "code":    "FILE_TOO_LARGE",
    "message": "File exceeds the 10MB limit."
  }
}
```

**Error -- 415 Unsupported Media Type**

```
{
  "error": {
    "code":    "UNSUPPORTED_FILE_TYPE",
    "message": "Only PNG, JPG, and GIF files are accepted."
  }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 201 | Created | Upload successful. Cloudinary URL returned. |

| | | |
|---|---|---|
| 400 | Bad Request | No file attached, or file exceeds 10MB. |
| 401 | Unauthorized | No valid JWT. |
| 415 | Unsupported Media Type | File type not PNG, JPG, or GIF. |

# Section 6: Comment Endpoints

Comments live on the gem detail page. The client fetches them paginated and submits new ones inline.

| GET | /gems/:id/comments | List comments for a gem | Auth required |
|---|---|---|---|

Purpose: The gem detail page renders the comment thread. Comments are returned in chronological order. The is_author flag tells the front-end whether to render the "Author" badge on each comment.

### Query parameters

```
page=1     default 1
limit=5    default 5
```

### Success response -- 200 OK

```
{
  "comments": [
    {
      "id":          "e5f6g7h8-...",
      "body":        "This looks absolutely stunning! How difficult is the
trail?",
      "is_author":   false,
      "created_at":  "2026-03-10T15:00:00Z",
      "author": {
        "username":    "sarah_adventures",
        "display_name":"Sarah",
        "avatar_url":  "https://res.cloudinary.com/..."
      }
    }
  ],
  "pagination": { "page": 1, "limit": 5, "total": 8 }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Comment list returned. |
| 404 | Not Found | Gem does not exist. |

| POST | /gems/:id/comments | Post a new comment | Auth required |
|---|---|---|---|

Purpose: Maya types a comment on the gem detail page and clicks "Post Comment". The new comment appears at the bottom of the thread immediately.

### Request body

```
{ "body": "Added this to my must-visit list! Thanks for the exact location." }
```

### Success response -- 201 Created

```
{
  "id":        "f6g7h8i9-...",
  "body":      "Added this to my must-visit list! Thanks for the exact location.",
  "is_author": false,
  "created_at":"2026-03-22T10:30:00Z",
  "author": {
    "username":    "mayachen",
    "display_name":"Maya Chen",
    "avatar_url":  "https://res.cloudinary.com/..."
  }
}
```

### Error -- 400 (empty body)

```
{
  "error": {
    "code":    "VALIDATION_ERROR",
    "message": "Request body failed validation.",
    "fields":  { "body": "Comment cannot be empty." }
  }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 201 | Created | Comment posted. |
| 400 | Bad Request | Empty body or body exceeds character limit. |
| 401 | Unauthorized | No valid JWT. |
| 403 | Forbidden | Comments disabled on this gem (privacy setting). |
| 404 | Not Found | Gem does not exist. |

# Section 7: Save (Bookmark) Endpoints

Saves implement the bookmark feature. A user can save any gem to their personal list, and unsave it later. The saved list is viewable on the profile page under the Saved tab.

| POST | /gems/:id/saves | Save (bookmark) a gem | Auth required |
|---|---|---|---|

Purpose: Maya taps the bookmark icon on a gem she wants to revisit. This inserts a row into the saves table. If the gem is already saved, a 409 is returned.

**Request body**

```
(empty body)
```

**Success response -- 201 Created**

```
{ "message": "Gem saved.", "save_count": 25 }
```

**Error -- 409 Already saved**

```
{ "error": { "code": "ALREADY_SAVED", "message": "You have already saved this
gem." } }
```

| Status | Meaning | When returned |
|---|---|---|
| 201 | Created | Gem saved. |
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | Gem does not exist. |
| 409 | Conflict | Gem already saved by this user. |

| DELETE | /gems/:id/saves | Unsave (remove bookmark) | Auth required |
|---|---|---|---|

Purpose: Maya taps the bookmark icon again on a gem she has already saved. This deletes the row from the saves table.

**Success response -- 200 OK**

```
{ "message": "Gem unsaved.", "save_count": 24 }
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Gem unsaved. |

| | | |
|---|---|---|
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | Gem not found or was not saved by this user. |

| GET | /users/me/saves | Get current user's saved gems | Auth required |
|---|---|---|---|

Purpose: The Saved tab on the user profile page loads Maya's personal bookmarked gems list, sorted by most recently saved.

**Success response -- 200 OK**

```
{
  "gems": [
    {
      "id":          "c3d4e5f6-...",
      "name":        "Hidden Cascade Behind the Old Mill",
      "cover_photo": "https://res.cloudinary.com/...",
      "category":    "Nature",
      "save_count":  24,
      "saved_at":    "2026-03-21T09:00:00Z"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 12 }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Saved gem list returned. |
| 401 | Unauthorized | No valid JWT. |

# Section 8: Follow Endpoints (Stretch Feature)

Follows implement the social graph. Following a user causes their new public gems to appear in the follower's personalized feed (GET /gems?feed=following). The Community page uses these endpoints to render Follow and Following button states.

**POST** `/users/:username/follows` Follow a user Auth required

Purpose: Sofia clicks "Follow" on James's card in the Community page. This inserts a row into the follows table with follower_id = Sofia and following_id = James.

### Request body

```
(empty body)
```

### Success response -- 201 Created

```
{
  "message":        "Now following jamesokafor.",
  "follower_count": 1241
}
```

### Error -- 409 Already following

```
{ "error": { "code": "ALREADY_FOLLOWING", "message": "You are already following
this user." } }
```

### Error -- 400 Cannot follow yourself

```
{ "error": { "code": "CANNOT_FOLLOW_SELF", "message": "You cannot follow
yourself." } }
```

| Status | Meaning | When returned |
|---|---|---|
| 201 | Created | Follow relationship created. |
| 400 | Bad Request | Attempting to follow self. |
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | Target user does not exist. |
| 409 | Conflict | Already following this user. |

**DELETE** `/users/:username/follows` Unfollow a user Auth required

Purpose: Sofia clicks the "Following" button again to unfollow James. This deletes the row from the follows table.

**Success response -- 200 OK**

```
{
  "message":        "Unfollowed jamesokafor.",
  "follower_count": 1240
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Follow relationship deleted. |
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | Target user does not exist or was not being followed. |

| GET | /users/:username/followers | List a user's followers | Auth required |
|---|---|---|---|

Purpose: The Followers tab on the Community page lists all users who follow the current user. Each card includes whether the current user follows them back (for Follow/Following button state).

**Success response -- 200 OK**

```
{
  "followers": [
    {
      "username":      "sofia_ramirez",
      "display_name":  "Sofia Ramirez",
      "avatar_url":    "https://res.cloudinary.com/...",
      "bio":           "Weekend explorer.",
      "gem_count":     14,
      "follower_count":320,
      "is_following":  true,
      "created_at":    "2025-06-01T00:00:00Z"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1241 }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Follower list returned. |
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | User does not exist. |

| GET | /users/:username/following | List users a user follows | Auth required |
|---|---|---|---|

Purpose: The Following tab on the Community page lists all users that the current user follows.

**Success response -- 200 OK (same shape as /followers)**

```
{
  "following": [
    {
      "username":      "jamesokafor",
      "display_name":  "James Okafor",
      "avatar_url":    "https://res.cloudinary.com/...",
      "gem_count":     127,
      "follower_count":1241,
      "is_following":  true
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 89 }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Following list returned. |
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | User does not exist. |

# Section 9: User Gem List Endpoint

Returns the gems submitted by a specific user. Used to populate the My Gems tab on the profile page.

| GET | /users/:username/gems | List gems submitted by a user | Auth required |
|---|---|---|---|

Purpose: The My Gems tab on James's profile page calls this endpoint with his username. For James viewing his own profile, private gems are included. For other users viewing his profile, only public gems (and followers_only gems if the viewer follows James) are returned.

**Success response -- 200 OK**

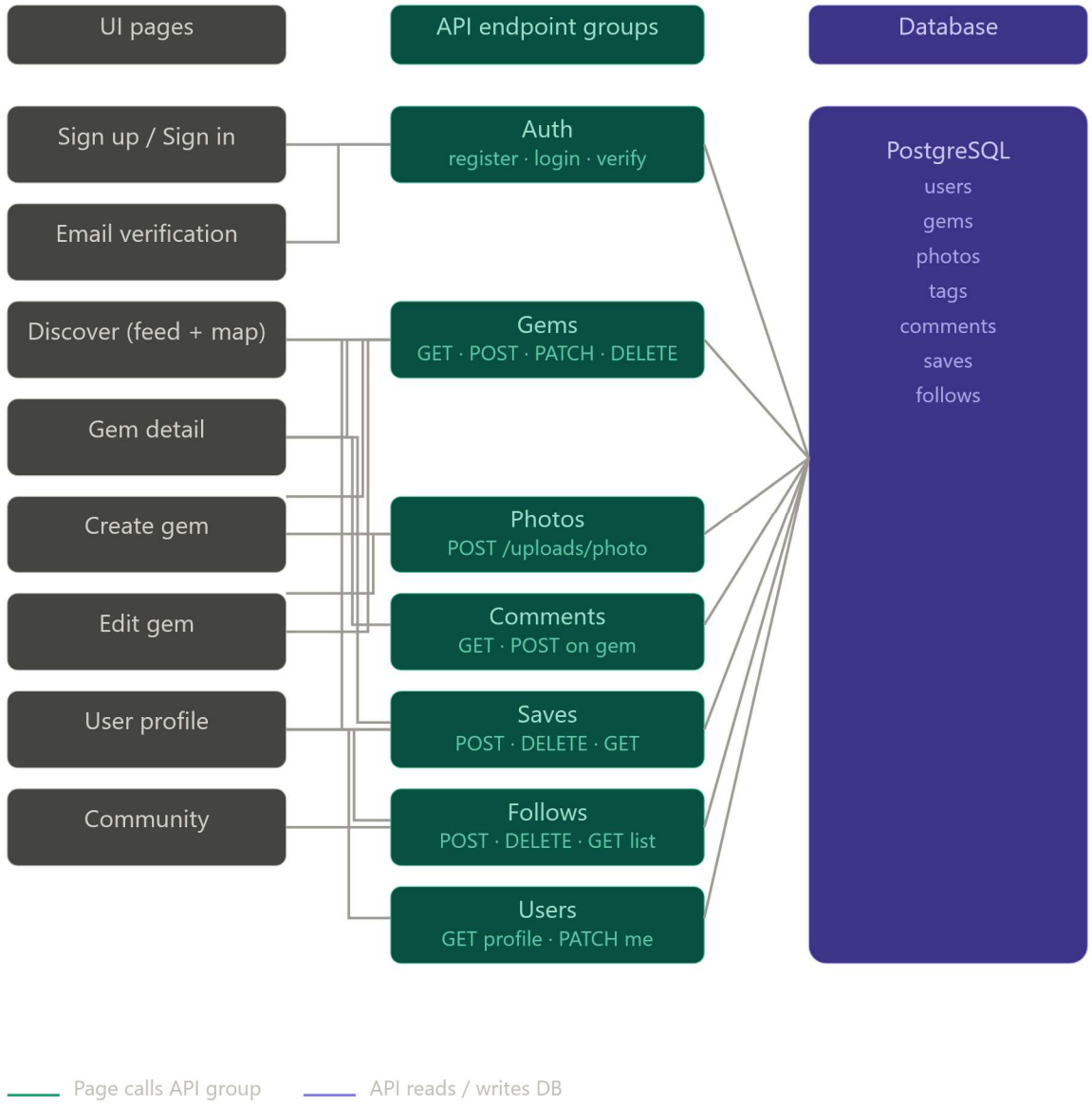
```
{
  "gems": [
    {
      "id":          "d4e5f6g7-...",
      "name":        "Secret Waterfall Trail",
      "category":    "Nature",
      "cover_photo": "https://res.cloudinary.com/...",
      "save_count":  18,
      "tags":        ["waterfall", "hiking"],
      "created_at":  "2026-03-10T14:30:00Z"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 127 }
}
```

| Status | Meaning | When returned |
|---|---|---|
| 200 | OK | Gem list returned. |
| 401 | Unauthorized | No valid JWT. |
| 404 | Not Found | User does not exist. |

# Section 10: Page-to-Endpoint Mapping

The following table maps each UI page to the service endpoints it calls. This confirms that the API covers every user interaction in the MVP and contains no endpoints that are unused by the interface.

| UI page | Endpoint(s) called |
|---|---|
| **Sign Up (/signup)** | `POST /auth/register` |
| **Sign In (/signin)** | `POST /auth/login` |
| **Email Verification (/verify-email)** | `POST /auth/verify-email`<br>`POST /auth/resend-verification` |
| **Discover -- Feed view (/discover)** | `GET /gems`<br>`GET /gems?feed=following (when Following filter active)`<br>`GET /gems?category=X&tag=Y&lat=...&lng=...&radius=... (filters)`<br>`POST /gems/:id/saves (bookmark from card)`<br>`DELETE /gems/:id/saves (unbookmark from card)` |
| **Discover -- Map view (/discover?view=map)** | `GET /gems (same params as feed, used for pin layer)` |
| **Gem Detail (/gems/:id)** | `GET /gems/:id`<br>`GET /gems/:id/comments`<br>`POST /gems/:id/comments`<br>`POST /gems/:id/saves`<br>`DELETE /gems/:id/saves`<br>`POST /gems/:id/flag` |
| **Create Gem (/create)** | `POST /uploads/photo (for each photo selected)`<br>`POST /gems` |
| **Edit Gem (/gems/:id/edit)** | `GET /gems/:id (pre-fill form)`<br>`POST /uploads/photo (if changing photo)`<br>`PATCH /gems/:id`<br>`DELETE /gems/:id` |
| **User Profile (/profile/:username)** | `GET /users/:username`<br>`GET /users/:username/gems (My Gems tab)`<br>`GET /users/me/saves (Saved tab, own profile only)`<br>`POST /users/:username/follows`<br>`DELETE /users/:username/follows`<br>`PATCH /users/me (Edit Profile, own profile only)`<br>`POST /uploads/photo (avatar change, own profile only)` |
| **Community (/community)** | `GET /users/:username/followers (Followers tab)`<br>`GET /users/:username/following (Following tab)`<br>`POST /users/:username/follows`<br>`DELETE /users/:username/follows` |


UI pages
API endpoint groups
Database
Sign up / Sign in
Email verification
Discover (feed + map)
Gem detail
Create gem
Edit gem
User profile
Community
Auth
register · login · verify
Gems
GET · POST · PATCH · DELETE
Photos
POST /uploads/photo
Comments
GET · POST on gem
Saves
POST · DELETE · GET
Follows
POST · DELETE · GET list
Users
GET profile · PATCH me
PostgreSQL
users
gems
photos
tags
comments
saves
follows
Page calls API group
API reads / writes DB

End